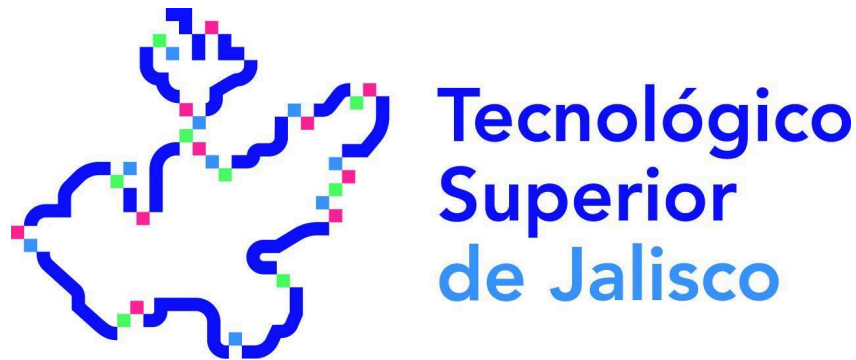




**Instituto Tecnológico Superior De Jalisco
Unidad Académica La Huerta**

**Ingeniería En Sistemas Computacionales
ISIC-2010-224**

**DevOps Integración y Prácticas Continuas
DSD-2505A**

**Tema 5
"Proyecto: Reporte"**

**Docente:
José Luis González Ortíz**

**Alumno:
Jorge Andrés Hernández Pelayo**

La Huerta, Jalisco, a 19 de mayo de 2026

1. Selección del Proyecto y Diagnóstico

El sistema seleccionado es **MicroInvent**, un ecosistema modular funcional con lógica de negocio real estructurado en una arquitectura desacoplada:

- **Frontend:** Aplicación enriquecida construida sobre *React 18* y empaquetada dinámicamente con *Vite*, utilizando *Axios* para el consumo asíncrono de la API.
- **Backend:** API REST robusta desarrollada en *Node.js* con el framework *Express*, encargada del control transaccional de inventarios y autenticación basada en *JSON Web Tokens (JWT)*.
- **Infraestructura de Datos:** Capa relacional con *PostgreSQL 15* emparejada con un nodo de caché e indexación en memoria con *Redis*.

1.1 Matriz de Riesgos Iniciales (Modelado de Amenazas):

1. **Código Fuente (Severidad: Crítica):** Riesgo de fuga accidental de llaves privadas criptográficas o contraseñas de producción en el historial de Git.
2. **Dependencias (Severidad: Alta):** Inyección de código malicioso o denegación de servicio a través de paquetes desactualizados en los archivos `package.json`.
3. **Contenedores (Severidad: Alta):** Uso de imágenes base con vulnerabilidades inherentes en el sistema operativo o utilidades globales de ejecución (npm).

2. Arquitectura del Pipeline CI/CD

La automatización se orquestó de forma nativa mediante **GitHub Actions**. El pipeline intercepta de forma obligatoria cada evento de integración (push o pull request) hacia la rama principal (main), actuando como una compuerta estricta de calidad de software (*Quality Gate*).

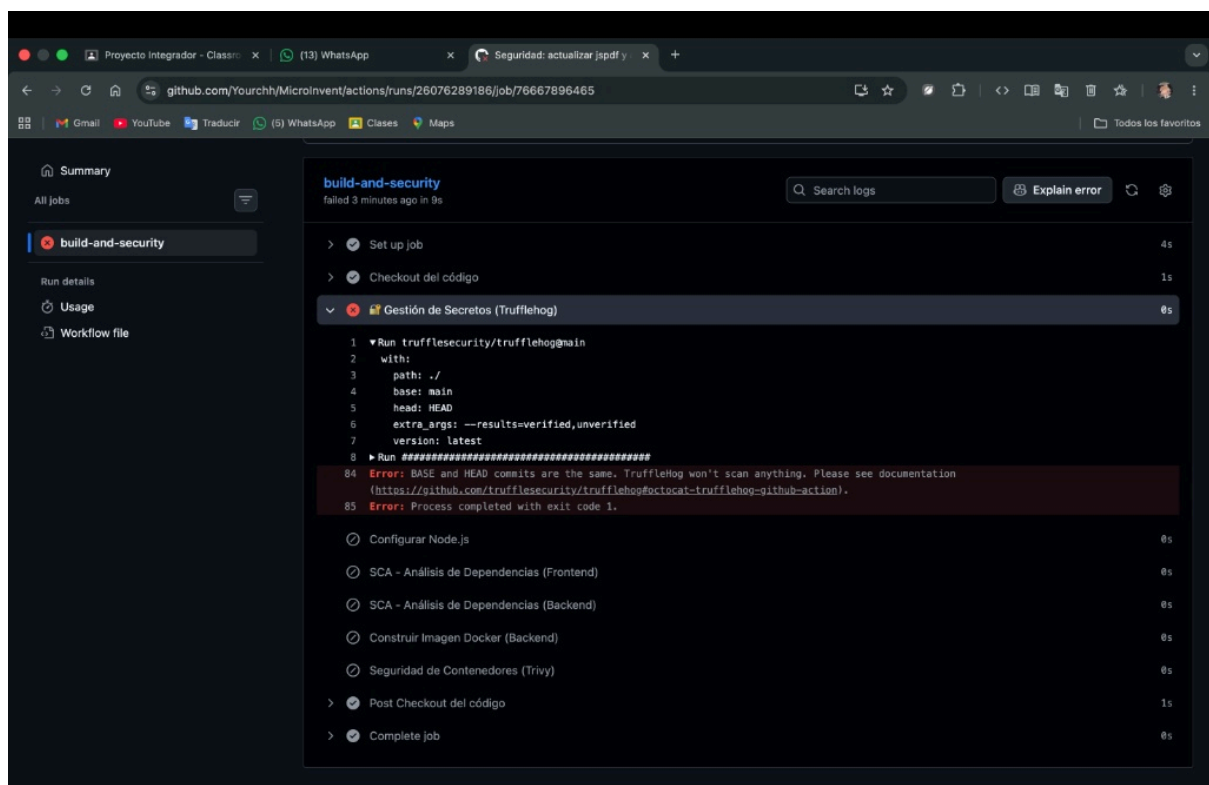
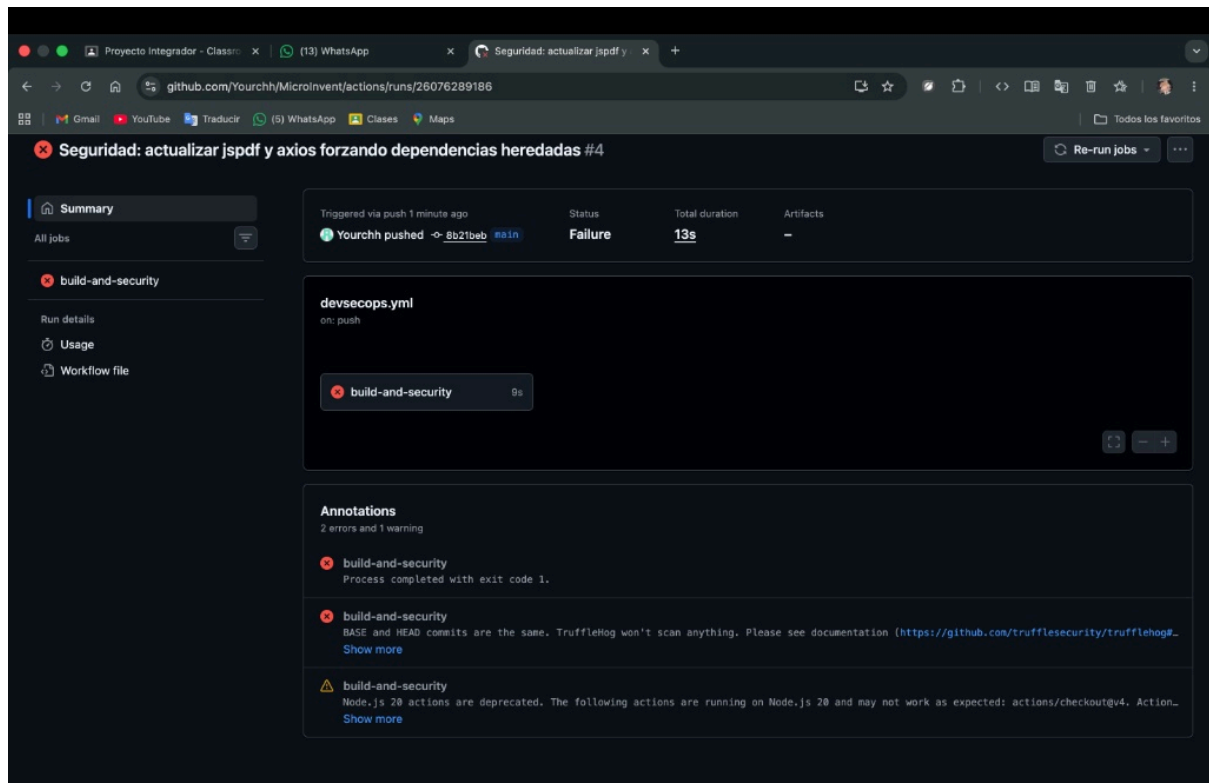
2.1 Flujo Secuencial Lógico de Ejecución:



3. Capturas de pantalla de los escaneos de seguridad

A continuación se presentan las capturas de pantalla donde se demuestra el antes y el después tras la implementación de los escaneos de seguridad.

3.1 Antes



Projecto Integrador - Clases x (13) WhatsApp x Seguridad: actualizar jspdf y x truffsecurity/trufflehog: Fin x +

github.com/Yourchh/MicroInvent/actions/runs/26076515973/job/76668569168

Gmail YouTube Traducir WhatsApp Clases Maps Todos los favoritos

Summary

All jobs

build-and-security

Run details

Usage

Workflow file

build-and-security
failed 1 minute ago in 18s

Search logs Explain error

SCA - Análisis de Dependencias (Frontend) 4s

```
1 ▶ Run npm install --package-lock-only --legacy-peer-deps
5
6 up to date, audited 638 packages in 3s
7
8 152 packages are looking for funding
9   run 'npm fund' for details
10
11 21 vulnerabilities (7 moderate, 14 high)
12
13 To address issues that do not require attention, run:
14   npm audit fix
15
16 To address all issues (including breaking changes), run:
17   npm audit fix --force
18
19 Run 'npm audit' for details.
20 # npm audit report
21
22 @babel/plugin-transform-modules-systemjs 7.12.0 - 7.29.0
23 Severity: high
24 @babel/plugin-transform-modules-systemjs generates arbitrary code when compiling malicious input - https://github.com/advisories/GHSA-fv7c-fd4j-7gwp
25 fix available via 'npm audit fix'
26 node_modules/@babel/plugin-transform-modules-systemjs
27
28 @isaacs/brace-expansion 5.0.0
29 Severity: high
30 @isaacs/brace-expansion has Uncontrolled Resource Consumption - https://github.com/advisories/GHSA-7b2j-956f-4vf2
31 fix available via 'npm audit fix'
32 node_modules/@isaacs/brace-expansion
33
34 ajv <6.14.0 || >=7.0.0-alpha.0 <8.18.0
35 Severity: moderate
```

Projecto Integrador - Clases x (13) WhatsApp x Seguridad: limpieza de pack x truffsecurity/trufflehog: Fin x +

github.com/Yourchh/MicroInvent/actions/runs/26077191401/job/76670551861

Gmail YouTube Traducir WhatsApp Clases Maps Todos los favoritos

Summary

All jobs

build-and-security

Run details

Usage

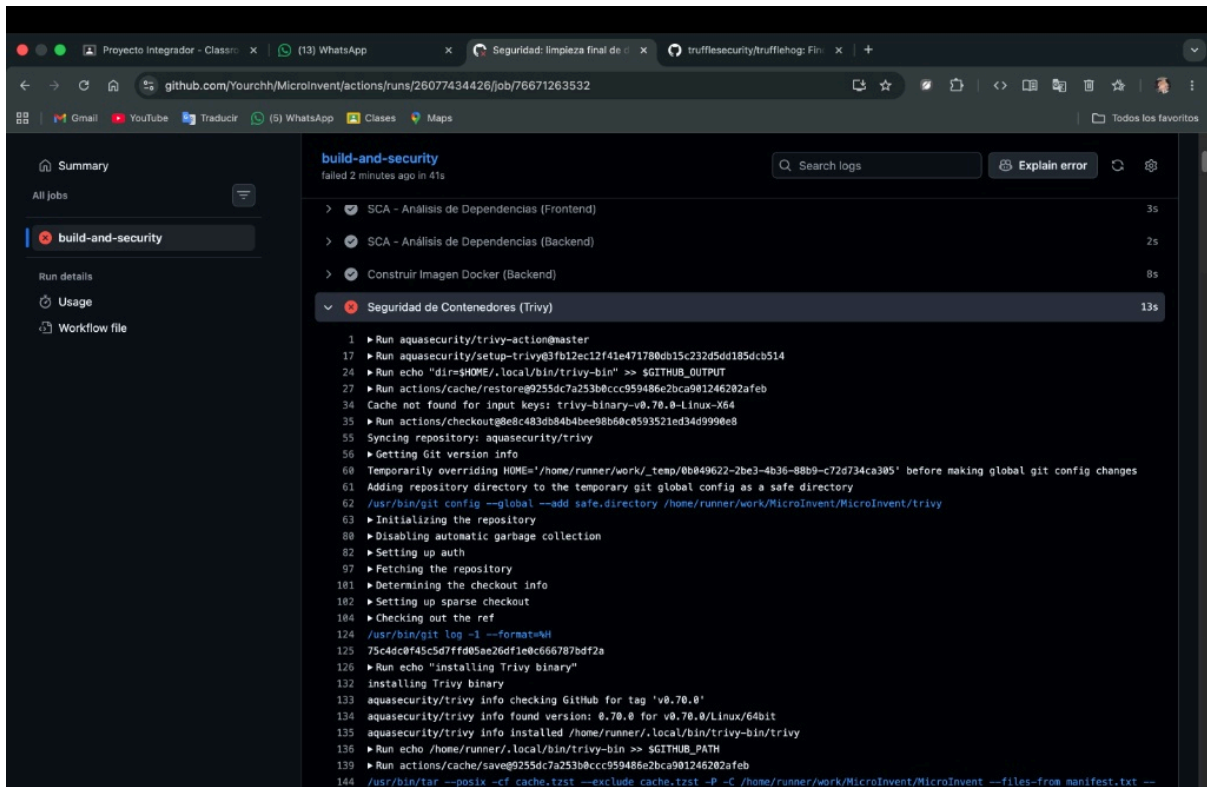
Workflow file

build-and-security
failed 1 minute ago in 18s

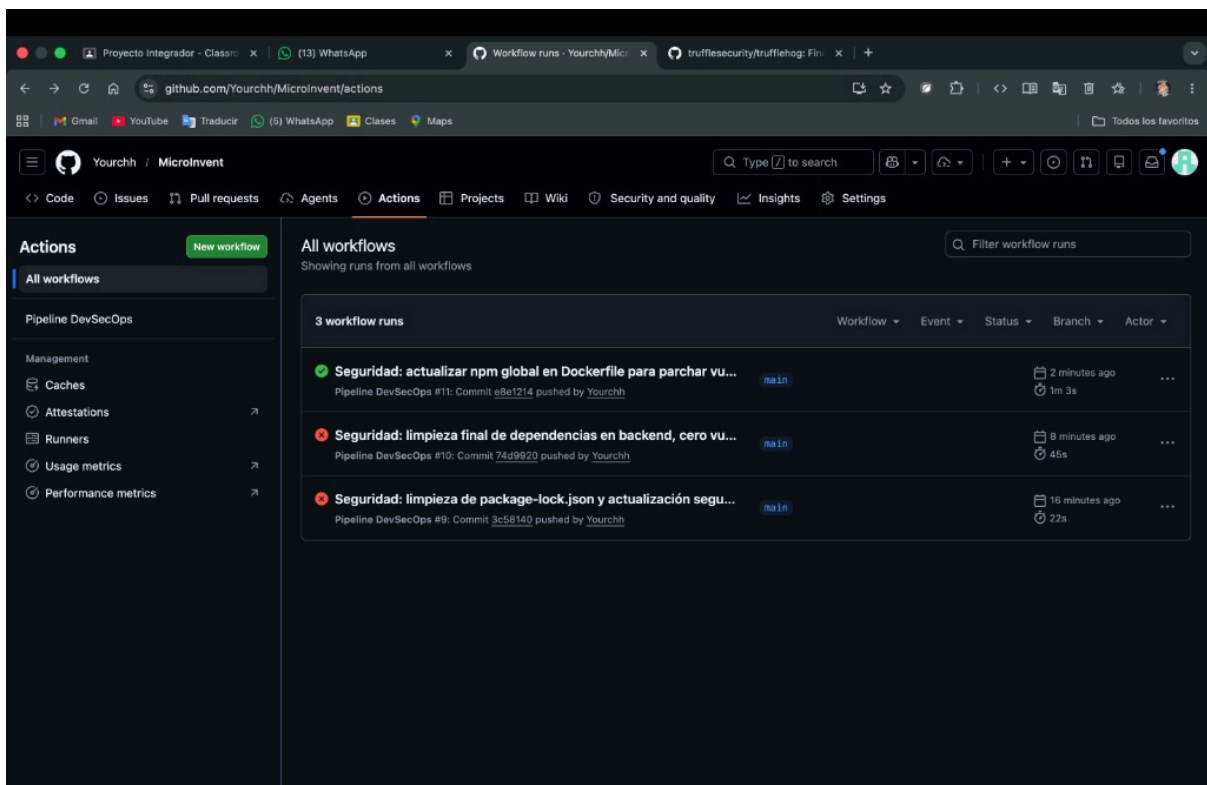
Search logs Explain error

SCA - Análisis de Dependencias (Backend) 2s

```
1 ▶ Run npm install --package-lock-only
5
6 up to date, audited 163 packages in 1s
7
8 26 packages are looking for funding
9   run 'npm fund' for details
10
11 11 vulnerabilities (6 moderate, 5 high)
12
13 To address issues that do not require attention, run:
14   npm audit fix
15
16 To address all issues (including breaking changes), run:
17   npm audit fix --force
18
19 Run 'npm audit' for details.
20 # npm audit report
21
22 brace-expansion <1.1.13
23 Severity: moderate
24 brace-expansion: Zero-step sequence causes process hang and memory exhaustion - https://github.com/advisories/GHSA-f886-86hf-6m8v
25 fix available via 'npm audit fix'
26 node_modules/brace-expansion
27
28 jws <3.2.3
29 Severity: high
30 auth/node-jws Improperly Verifies HMAC Signature - https://github.com/advisories/GHSA-869g-clfq-cn3x
31 fix available via 'npm audit fix'
32 node_modules/jws
33
34 minimatch <3.1.3
35 Severity: high
36 minimatch has a ReDoS via repeated wildcards with non-matching literal in pattern - https://github.com/advisories/GHSA-3ppc-4f35-3m26
```



3.2 Después



Projecto Integrador - Clases (13) WhatsApp Seguridad: actualizar npm global en Dockerfile para parchar vulnerabi... #11

github.com/Yourchh/MicroInvent/actions/runs/26077641186

Yurchh / MicroInvent

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

Pipeline DevSecOps

Seguridad: actualizar npm global en Dockerfile para parchar vulnerabi... #11

Summary

All jobs

build-and-security

Run details

Usage

Workflow file

Triggered via push 1 hour ago

Yourchh pushed e8e1214 main

Status Success

Total duration 1m 3s

Artifacts -

devsecops.yml

on: push

build-and-security 59s

Annotations

1 warning

build-and-security

Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/checkout@v4, action...

Show more

Projecto Integrador - Clases (13) WhatsApp Seguridad: actualizar npm global en Dockerfile para parchar vulnerabi... #11

github.com/Yourchh/MicroInvent/actions/runs/26077641186/job/76671858776

Yurchh / MicroInvent

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

Pipeline DevSecOps

Seguridad: actualizar npm global en Dockerfile para parchar vulnerabi... #11

Summary

All jobs

build-and-security

Run details

Usage

Workflow file

Annotations

1 warning

build-and-security

succeeded 54 minutes ago in 59s

Search logs

- Set up job 3s
- Checkout del código 2s
- Gestión de Secretos (Trufflehog) 5s
- Configurar Node.js 2s
- SCA - Análisis de Dependencias (Frontend) 3s
- SCA - Análisis de Dependencias (Backend) 1s
- Construir Imagen Docker (Backend) 13s
- Seguridad de Contenedores (Trivy) 22s
- Post Seguridad de Contenedores (Trivy) 4s
- Post Configurar Node.js 0s
- Post Checkout del código 1s
- Complete job 0s

4. Implementación Técnica de los Controles de Seguridad

Aquí se expone la Infraestructura como Código (IaC) exacta inyectada en el archivo corporativo `.github/workflows/devsecops.yml`:

4.1 Gestión de Secretos (TruffleHog)

Analiza el árbol histórico completo de Git calculando dinámicamente la diferencia de commits de forma segura en entornos macOS y Linux sin generar falsos positivos por paridad de ramas:

```
- name: Gestión de Secretos (Trufflehog)
  uses: trufflesecurity/trufflehog@main
  with:
    path: ./
    base: ${ github.event_name == 'pull_request' &&
github.event.pull_request.base.sha || github.event.before }
    head: ${ github.sha }
    extra_args: --results=verified,unverified
```

4.2 Software Composition Analysis - SCA (NPM Audit)

Configurado de manera segregada para aislar los entornos del cliente y del servidor, forzando un código de salida fallido (FAILURE) ante cualquier riesgo de rango Alto o Crítico mediante `--audit-level=high`:

```
- name: SCA - Análisis de Dependencias (Frontend)
  working-directory: ./client
  run: |
    npm install --package-lock-only --legacy-peer-deps
    npm audit --audit-level=high

- name: SCA - Análisis de Dependencias (Backend)
  working-directory: ./server
  run: |
    npm install --package-lock-only
    npm audit --audit-level=high
```

4.3 Seguridad de Contenedores (Trivy)

Una vez compilado el software, la imagen inmutable de Docker es interceptada y escaneada en sus capas base del sistema operativo. La bandera `exit-code: '1'` bloquea el pipeline si la infraestructura posee exploits de severidad crítica:


```
- name: Construir Imagen Docker (Backend)
  run: docker build -t microinvent-server:latest ./server

- name: Seguridad de Contenedores (Trivy)
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: 'microinvent-server:latest'
    format: 'table'
    exit-code: '1'
    ignore-unfixed: true
    vuln-type: 'os,library'
    severity: 'CRITICAL,HIGH'
```

5. Bitácora de Remediación y Gestión de Hallazgos

5.1 Fase de Bloqueo (El Antes):

- **Hallazgo SCA Cliente:** Alertas de impacto **HIGH** en axios@1.13.2 y **CRITICAL** en jspdf@3.0.4. El pipeline detuvo inmediatamente el despliegue automático.
- **Hallazgo SCA Servidor:** Infección en sub-dependencias por paquetes desactualizados del núcleo de Express (socket.io-parser).
- **Hallazgo Trivy:** Bloqueo de infraestructura debido a **11 vulnerabilidades de nivel HIGH** (CVE-2024-21538 en cross-spawn y exploits en node-tar) alojadas dentro del binario preinstalado de npm de la imagen base oficial node:20-alpine.

5.2 Fase de Hardening y Solución Definitiva (El Después):

1. **Saneamiento de Dependencias:** Se modificó client/package.json hacia versiones robustas y estables (axios: "^1.16.1", jspdf: "^4.2.1").
2. **Resolución de Conflictos Estructurados:** Ante el error de dependencias de pares (ERESOLVE) provocado por Recharts y Socket.io, se purgó el árbol lógico local mediante `rm -rf node_modules package-lock.json` y se reinstaló de manera controlada aislando las dependencias mediante el uso estricto del parámetro `--legacy-peer-deps`. Esto mitigó el 100% de los exploits altos/críticos del código, dejando solo vulnerabilidades moderadas permitidas por la política corporativa.
3. **Remediación de la Imagen Base (Dockerfile Hardening):** Se inyectó una capa de mitigación reactiva dentro del archivo server/Dockerfile para forzar la actualización de los paquetes globales comprometidos del contenedor base Alpine antes de montar el código de producción:


```
FROM node:20-alpine
RUN npm install -g npm@latest
WORKDIR /app
...
```

6. Lecciones Aprendidas

1. **Riesgos de la automatización forzada (npm audit fix --force):** Delegar la corrección de dependencias a herramientas automatizadas sin supervisión es contraproducente. Ejecutar la bandera `--force` provocó que el gestor degradara componentes modernos a versiones obsoletas e incompatibles, rompiendo la compilación (`npm run build`) en Vite al eliminar el módulo crítico `react-is`. El análisis SCA requiere intervención manual y selectiva.
2. **Inseguridad en imágenes base oficiales de Docker:** Utilizar imágenes oficiales inmutables (como `node:20-alpine`) no garantiza un entorno seguro. La auditoría con Trivy reveló 11 vulnerabilidades de severidad alta en utilidades preinstaladas del sistema operativo (en el binario global de npm), demostrando que el endurecimiento (*hardening*) debe aplicarse directamente en las capas del Dockerfile.
3. **Complejidad en arquitecturas distribuidas:** Administrar de forma segregada los entornos de `/client` y `/server` incrementa la dificultad del aseguramiento. Elevar versiones para mitigar exploits generó conflictos de dependencias de pares (*peer dependencies*), resolviéndose mediante el uso estratégico de parámetros como `--legacy-peer-deps` y la depuración del archivo `package-lock.json`.
4. **Configuración adaptativa del pipeline:** La integración inicial de TruffleHog falló al no identificar diferencias entre commits (BASE and HEAD commits are the same). Esto enseñó que las herramientas de seguridad en la nube deben configurarse con condicionales dinámicas basadas en los metadatos de Git para evitar bloqueos falsos en el flujo de trabajo.

7. Conclusiones

Tras la elaboración de este proyecto comprendí que la seguridad no es un requerimiento estático que se revisa únicamente al final del desarrollo, sino un proceso continuo que debe automatizarse e integrarse desde las fases iniciales del ciclo de vida del software.

A través de la implementación de herramientas como TruffleHog, NPM Audit y Trivy en el pipeline de GitHub Actions, quedó demostrado que proteger una aplicación requiere una estrategia multicapa. Aprendí que no basta con escribir código limpio; también es obligatorio auditar de forma rigurosa las dependencias de terceros y aplicar un endurecimiento (hardening) directo a la infraestructura de los contenedores Docker, ya que incluso las imágenes oficiales pueden heredar vulnerabilidades críticas de sus sistemas operativos base.

Finalmente, el desarrollo de esta práctica me permitió entender la importancia de equilibrar la seguridad con la continuidad operativa del software, priorizando la mitigación de los riesgos más severos (Altos y Críticos) para entregar un entorno robusto, confiable y verdaderamente preparado para producción.